

Informatika 3

Prednáška 2: Typy a ich reprezentácia

Michal Mrena



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Typové systémy



Typový systém

- Formálne sa nimi zaoberá teória typov
- Skupina pravidiel:
 - Vstavane typy, množina hodnôt, množina operácií, ...
 - Tvorba nových typov
 - Konverzie
- Rozlišujeme rôzne typové systémy z rôznych pohľadov
 - Statický / dynamický
 - Silne / slabo typovaný
 - Nominálny / štruktúrálly
 - ...

Dynamicky a staticky typované jazyky

Dynamicky

- Typová kontrola sa vykonáva **počas behu** programu
- Prípadná typová chyba sa prejaví **počas behu** programu
- Typ je spojený s **hodnotou**, nie s **premennou**

Staticky

- Typová kontrola sa vykonáva **pred spustením** programu
- Prípadná typová chyba sa prejaví **pri kompilácii**
- Typ je spojený s **premennou**, nie s **hodnotou**

Dynamicky a staticky typované jazyky

```
myvar = "Cormar nelde Eldatárin nu Tarmenel"  
myvar = 3
```

Python **OK**

```
String myvar;  
myvar = "Cormar nelde Eldatárin nu Tarmenel";  
myvar = 3;
```

Java **ERROR**
Main.java:11: error: incompatible types: `int` cannot
be converted to `String`
 myvar = 3;
 ^

```
var myvar =  
    "Cormar nelde Eldatárin nu Tarmenel";  
myvar = 3;
```

Java **ERROR**
Main.java:11: error: incompatible types: `int` cannot
be converted to `String`
 myvar = 3;
 ^

Dedukcia typov nemení statické typovanie

Statický typový systém

- Vhodné využitie silného statického typového systému umožňuje zachytiť logiku v samotnom typovom systéme, teda umožňuje:
 - urobiť nekonzistentný (chybný) stav programu **nereprezentovateľným**,
 - urobiť chybné správanie **typovou chybou**
- Správnosť programu sa do určite miery kontroluje už **pri kompilácii** (pred nasadením!)
- Ben Deane – Using Types Effectively:
 - <https://www.youtube.com/watch?v=ojZbFIQSdl8>

Slabo a silne typované jazyky

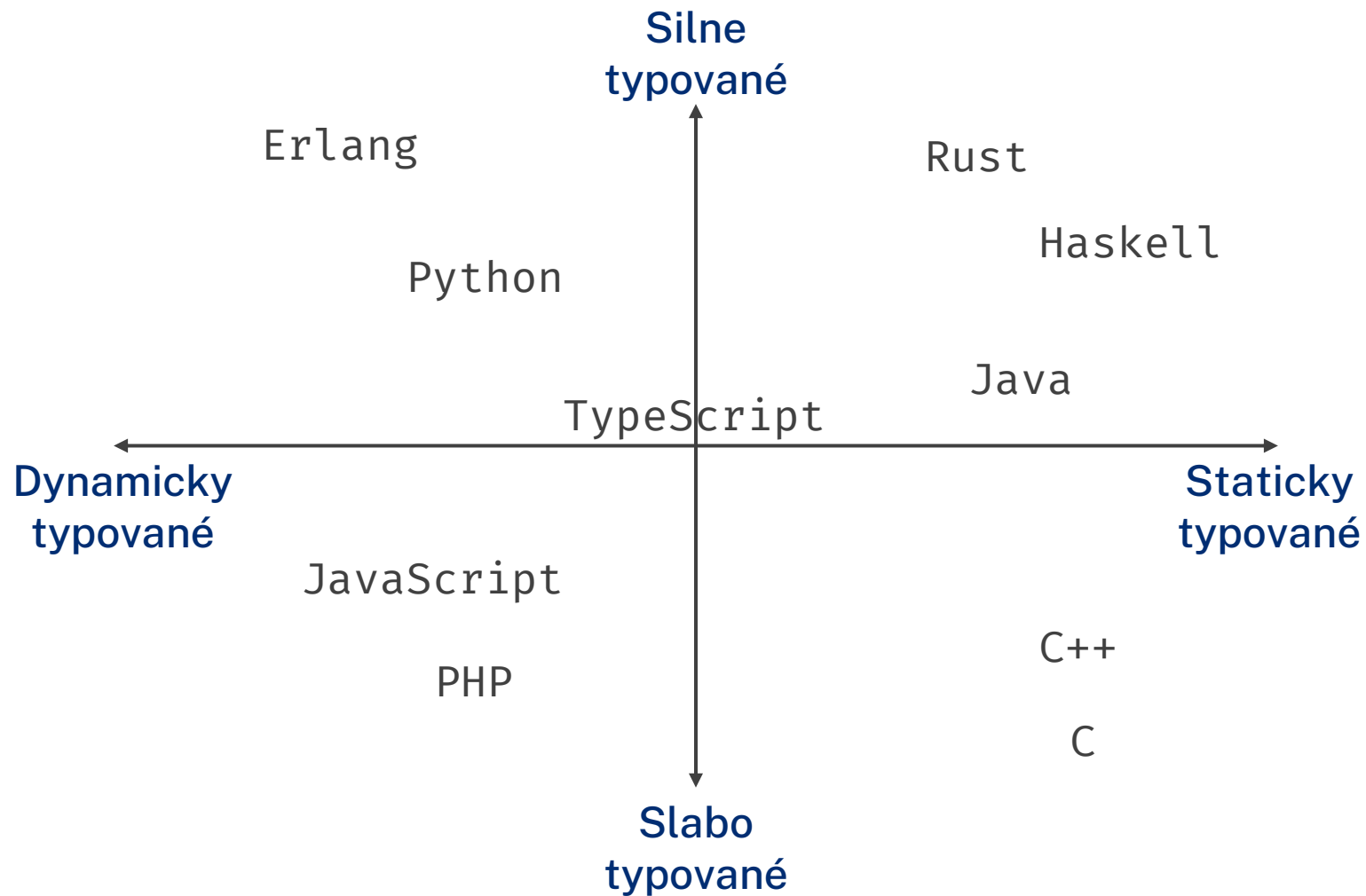
- Jednoznačná definícia neexistuje
- *Sila* hovorí o miere v akej typový systém pripúšťa **implicitné konverzie** (napr. tzv. type coercion v jazyku JavaScript)

```
s = "hello" + 5  
print(s)
```

```
Python ERROR  
Traceback (most recent call last):  
  File "main.py", line 2, in <module>  
    str = "hello" + 5  
TypeError: unsupported operand type(s) for +:  
'int' and 'str'
```

```
let s = "5" - "2";  
console.log(s);
```

```
JavaScript OK  
3
```



* ! Obrázok je ilustratívny, nesnaží sa implikovať, že dynamickosť a sila typovania sú objektívne kvantifikovateľné. Relatívne usporiadanie jazykov tiež nie je objektívne.

Duck typing

- Dynamický typový systém, v ktorom, ak objekt má metódu s daným názvom schopnú akceptovať daný paramter(e), tak sa metóda zavolá, bez ohľadu na skutočný typ objektu.

```
class Cat {  
    doSomething() {  
        console.log("Where is my food?");  
    }  
}  
  
class Dog {  
    doSomething() {  
        console.log("Woof! Woof!");  
    }  
}  
  
function test(something) {  
    something.doSomething();  
}  
  
cat = new Cat();  
dog = new Dog();  
test(cat); // Where is my food?  
test(dog); // Woof! Woof!
```

Typy v jazyku C++

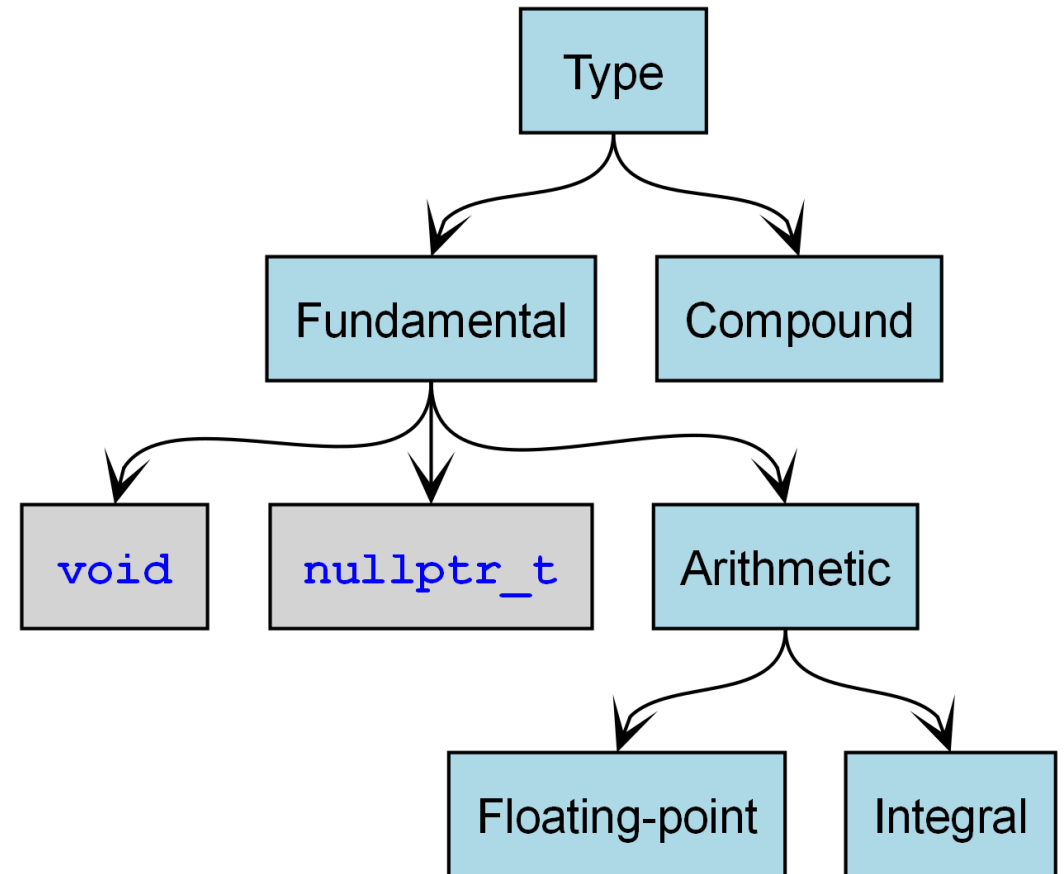
Fundamentálne typy v jazyku C++ a ich reprezentácia



Typy v C++

Objects, references, functions including function template specializations, and expressions have a property called **type**, which both restricts the **operations** that are permitted for those entities and provides semantic **meaning** to the otherwise generic **sequences of bits**.

Jazyk C++ je **staticky** **slabo** typovaný. V porovnaní s jazykom Java je typový systém rozsiahlejší a komplikovanejší.



Pamäť z pohľadu programátora

bit – najmenšia jednotka informácie – áno/nie, true/false, 0/1, ...

- angl. bit, skratka b

bajt – 8 bitov, najmenšia adresovateľná jednotka pamäte, $2^8 = 256$ možných hodnôt

- angl. byte, skratka B

pamäť – pracovná predstava – veľké pole bajtov, index bajtu budeme nazývať *adresa*

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]
67	111	114	109	97	114	32	110	101	108	100	101	0

Celočíselné typy (Integral types)

bool

- Nadobúda práve jednu z hodnôt `true` a `false`

char

- Slúži na reprezentáciu znakov*
- Nadobúda práve jednu z 256 hodnôt

int

- Základný celočíselný typ
- Za bežných okolností, použite `int`

```
int main()
{
    bool test = true;
    if (test) {
        doSomething();
    }

    char c = 'X';
    if (c == 'o') {
        doSomethingElse();
    }

    int x = 0;
    int y = x + 20;
}
```

Reprezentácia znakov a reťazcov je pomerne komplikovaná, bude jej venovaná časť inej prednášky. *

Modifikátory typu int

- Veľkosť typu je závislá na platforme, môže byť upravená *modifikátorom* veľkosti
- Znamienko
 - `signed` – umožňuje reprezentovať **záporné** a **nezáporné** hodnoty
 - `unsigned` – umožňuje reprezentovať **nezáporné** hodnoty
- Veľkosť
 - `short` – optimalizovaný pre veľkosť – aspoň 2 bajty
 - `long` – aspoň 4 bajty
 - `long long` – aspoň 8 bajtov
- Modifikátory sú voliteľné, ak sa žiadny neuvedie, predpokladá sa `signed` a žiadny modifikátor veľkosti
- Ak sa uvedie iba modifikátor, predpokladá sa typ `int`

Dátové modely

- Upravujú veľkosť typu `int`, vplyv modifikátora `long` a veľkosť *ukazovateľa**
- `long long int` má vždy aspoň 8B

LP32 `int` 2B, `long int` 4B, ukazovateľ 4B

ILP32 `int` 4B, `long int` 4B, ukazovateľ 4B

LLP64 `int` 4B, `long int` 4B, ukazovateľ 8B

LP64 `int` 4B, `long int` 8B, ukazovateľ 8B

Win16 API

32-bit Win32 API, 32bit UNIX

64-bit Win32 API

64-bit UNIX (Linux)

Viac o ukazovateľoch neskôr. *

Modifikátory typu int – prehľad

Zvyčajne píšeme	Presný význam	Typická veľkosť
short	signed short int	2B
unsigned short	unsigned short int	2B
unsigned	unsigned int	4B
long	signed long int	4B / 8B
unsigned long	unsigned long int	4B / 8B
long long	signed long long int	8B
unsigned long long	unsigned long long int	8B

Praktické overenie

- Operátor `sizeof` – vráti veľkosť argumentu (typu) v bajtoch

```
#include <iostream>
int main()
{
    std::cout << "short int      " << sizeof(short int)    << "\n";
    std::cout << "int          " << sizeof(int)          << "\n";
    std::cout << "long int     " << sizeof(long int)     << "\n";
    std::cout << "long long int" << sizeof(long long int) << "\n";
    std::cout << "void*        " << sizeof(void*)        << "\n";
}
```

short int	2
int	4
long int	4
long long int	8
void*	8

MSYS2-UCRT64, 64-bit Windows

short int	2
int	4
long int	8
long long int	8
void*	8

64-bit Linux

Reprezentácia celočíselných typov

- Uvažujme typ `int` o veľkosti 4B

- $1478958_{10} = 101101001000100101110_2 = 00000000000101101001000100101110_2$

```
int main() {  
    int prve = 16846;  
    int druhe = 1478958;  
    int tretie = prve + druhe;  
}
```

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]
00000000	00000000	01000001	11001110	00000000	00010110	10010001	00101110	00000000	00010110	11010010	11111100	00000000
prve				druhe				tretie				

Záporné celé čísla

- Na reprezentáciu celých čísel so znamienkom (**signed**) sa využíva tzv. dvojkový doplnok (Jon von Neumann, 1945)
- Záporné číslo reprezentuje ako kladnú vzdialenosť k **pomyselnej 0**
- Reprezentáciu záporného čísla získame:
 - **Invertovaním bitov**
 - **Pripočítaním 1**
- Pri záporných číslach má tak najvýznamnejší bit hodnotu 1
- Sčítanie a odčítanie funguje identicky

Získanie reprezentácie čísla
– 2 v jedno bajtovom type:

$$\begin{aligned} & -2_{10} \\ &= 0 - 2_{10} \\ &= 1\ 00000000_2 - 00000010_2 \\ &= 1 + 11111111_2 - 00000010_2 \\ &= 1 + 11111101_2 \\ &= 11111110_2 \end{aligned}$$

Rozsahy hodnôt

Veľkosť (B)	Počet hodnôt	Rozsah hodnôt	
		signed	unsigned
1	$2^{8 \cdot 1} = 2^8$	-128 127	0 255
2	$2^{8 \cdot 2} = 2^{16}$	-32 768 32 767	0 65 535
4	$2^{8 \cdot 4} = 2^{32}$	-2 147 483 648 2 147 483 647	0 4 294 967 295
8	$2^{8 \cdot 8} = 2^{64}$	-9 223 372 036 854 775 808 9 223 372 036 854 775 807	0 18 446 744 073 709 551 615

Typový alias

- Jazyk C++ umožňuje definovať alternatívny názov pre existujúci typ – **alias**
- Alias **nevytvára** nový typ – môžeme ho použiť všade, kde aj pôvodný typ
- Syntax:

`using <novy nazov> = <stary nazov>;`

```
#include <iostream>

int main()
{
    using myint = unsigned long long;
    myint x = 10;
    myint y = 20;
    myint z = x + y;
    std::cout << z << "\n";
}
```

Staršia syntax preberá z jazyka C.
Nebudeme ju používať, ale stretnete sa s ňou.

```
#include <iostream>

int main()
{
    typedef unsigned long long myint;
    myint x = 10;
    myint y = 20;
    myint z = x + y;
    std::cout << z << "\n";
}
```

Typový alias

- Alias môžeme zdefinovať v rozsahu platnosti funkcie/metódy, triedy, globálne
- Vždy vyberáme najmenší možný rozsah, vyhýbame sa globálnym aliasom (obzvlášť v hlavičkových súboroch)
- **Vhodné** využitie aliasu sprehľadňuje kód a uľahčuje prípadnú zmenu typu

```
class Player {  
public:  
    using ScoreType = long long;  
  
    ScoreType getScore() const;  
    void addScore(ScoreType value);  
    void removeScore(ScoreType value);  
  
private:  
    ScoreType m_score;  
};
```

Alias typov s pevnou veľkosťou

- Ak potrebujeme celočíselný typ s rovnakou veľkosťou na všetkých platformách, môžeme použiť štandardnú hlavičku <stdint>

```
#include <stdint>

class Mask {
public:
    bool isSet(int i) const;
    void set(int i);
    void unset(int i);

private:
    std::uint32_t m_bits;
};
```

Types

int8_t int16_t (optional) int32_t int64_t	signed integer type with width of exactly 8, 16, 32 and 64 bits respectively with no padding bits and using 2's complement for negative values (provided if and only if the implementation directly supports the type) (typedef)
int_fast8_t int_fast16_t int_fast32_t int_fast64_t	fastest signed integer type with width of at least 8, 16, 32 and 64 bits respectively (typedef)
int_least8_t int_least16_t int_least32_t int_least64_t	smallest signed integer type with width of at least 8, 16, 32 and 64 bits respectively (typedef)
intmax_t	maximum-width signed integer type (typedef)
intptr_t (optional)	signed integer type capable of holding a pointer to void (typedef)
uint8_t uint16_t (optional) uint32_t uint64_t	unsigned integer type with width of exactly 8, 16, 32 and 64 bits respectively (provided if and only if the implementation directly supports the type) (typedef)
uint_fast8_t uint_fast16_t uint_fast32_t uint_fast64_t	fastest unsigned integer type with width of at least 8, 16, 32 and 64 bits respectively (typedef)
uint_least8_t uint_least16_t uint_least32_t uint_least64_t	smallest unsigned integer type with width of at least 8, 16, 32 and 64 bits respectively (typedef)
uintmax_t	maximum-width unsigned integer type (typedef)
uintptr_t (optional)	unsigned integer type capable of holding a pointer to void (typedef)

Alias pre veľkosť

- Štandardná knižnica na niekoľkých miestach (napr. v hlavičke <cstdlib>) definuje alias std::size_t
- Ide o alias **unsigned** celočíselného typu, ktorý je schopný reprezentovať najväčšiu možnú veľkosť poľa/objektu na danej platforme
- Operátor **sizeof** vracia hodnotu tohto typu
- Na moderných platformách pôjde o **unsigned** 64 bitový **int**
- Používa sa ako **typ indexu** – aj keď sa toto rozhodnutie ukazuje ako nešťastné (viac tu, tu a tu), existujúci kód a štandardná knižnica sú mu nenapraviteľne prispôbené

Reálne typy (Floating-point types)

- Slúžia na reprezentáciu reálnych čísel (iba podmnožiny racionálnych čísel)

- C++ — ako väčšina jazykov — implementuje štandard [IEEE-754](#)

`float`

- [4B \(32 bitov\)](#)

`double`

- [8B \(64 bitov\)](#)

`long double`

- Môže byť iba (silný) alias `double` (MSVC)
- Môže mať [80 bitov](#)

Reálne typy – reprezentácia

- Podobná vedeckému zápisu dekadických čísel:
 - $271,828 = +0,271828 * 10^3$
- Ukladá:
 - **Znamienko** (1 bit)
 - **Exponent** (8 bitov, 11 bitov)
 - **Mantisu** (23 bitov, 52 bitov)

Príklad pre typ float:

$$0.15625_{10} = 00011111 | 00010000 | 00000000 | 000000_2$$
$$(-1)^{b_{31}} * 2^{(b_{30}b_{29}...b_{23})_2 - 127} * (1, b_{22}b_{21} ... b_0)_2$$

Reálne typy – presnosť

- Povaha reprezentácie implikuje limitovanú presnosť pri veľmi veľkých alebo malých hodnotách
- Problém je však aj v menej očividných prípadoch

```
#include <iostream>
int main() {
    double x = 0;
    for (int i = 0; i < 10; ++i) {
        x += 0.1;
    }
    if (x == 1.0) {
        std::cout << "ocakavam tento vypis" << "\n";
    } else {
        std::cout << "ale dostanem tento, x = " << x << "\n";
    }
}
```

```
$ ./src/main.exe
ale dostanem tento, x = 1
```

Reálne typy – presnosť

$$0,875 = 0,8 + 0,07 + 0,005 = \mathbf{8} * 10^{-1} + \mathbf{7} * 10^{-2} + \mathbf{5} * 10^{-3} = 0,875_{10}$$

$$0,875 = 0,1_2 + 0,01_2 + 0,001_2 = \mathbf{1} * 2^{-1} + \mathbf{1} * 2^{-2} + \mathbf{1} * 2^{-3} = 0,111_2$$

$$0,1 = 0,1_{10}$$

$$0,1 = \frac{1}{16} + \frac{1}{32} + \frac{1}{256} + \frac{1}{512} + \frac{1}{4096} + \frac{1}{8192} + \frac{1}{65536} + \dots = 0.0001100110011001101 \dots_2$$

Reálne typy – špeciálne hodnoty

- Špeciálne hodnoty **NaN** a **Infinity** môžu byť výsledkom operácií s reálnymi číslami
- Nedostávame tak napr. chybu pri delení nulou, ako pri celočíselných typoch

```
#include <iostream>
int main() {
    double x = 10;
    double y = 0;
    double z = x / y;
    double q = y / y;
    std::cout << z << "\n";
    std::cout << q << "\n";
}
```

```
$ ./src/main.exe
inf
nan
```

Výber číselného typu

- Ak je `int` postačujúci, použite (**long**) `int`
 - Ak naozaj potrebujete špecifické správanie `unsigned` modifikátora, použite ho
 - Ak naozaj potrebujete šetriť pamäť, použite `short` alebo `int8_t`
- Ak potrebujete bežné reálne čísla, použite `double`
 - Ak naozaj potrebujete šetriť miesto, použite `float`
- Ak potrebujete pokročilé optimalizácie (cache, SIMD), použite `float`
 - analytická geometria (grafika), strojové učenie, ...
- Ak potrebujete **presnosť** (napr. pri práci s peniazmi), vyhýbajte sa reálnym číslam
 - Niektoré jazyky poskytujú typ `decimal`

Kvalifikátory

- Typ je možné kvalifikovať kvalifikátormi `const` a `volatile`
- `volatile` na tomto predmete nebudeme potrebovať
- Ak vieme, že premenná sa nebude meniť, označíme jej typ kvalifikátorom `const`
- Vyjadrenie logiky v typovom systéme

```
#include <iostream>

int main() {
    const int mul = 10;
    int sum = 0;
    for (int i = 1; i < 10; ++i) {
        mul += i * mul;
    }
    std::cout << sum << "\n";
}
```

```
main.cpp: In function 'int main()':
main.cpp:7:13: error: assignment of read-only variable 'mul'
      7 |         mul += i * mul;
        |         ~~~~^~~~~~
make[2]: *** [src/CMakeFiles/main.dir/build.make:79:
src/CMakeFiles/main.dir/main.cpp.obj] Error 1
make[1]: *** [CMakeFiles/Makefile2:106:
src/CMakeFiles/main.dir/all] Error 2
make: *** [Makefile:91: all] Error 2
```

Endianita

- Vlastnosť CPU – určuje poradie bajtov v pamäti pre viac-bajtové typy
- Najčastejšie používané prístupy sú:
 - **little-endian** – najvýznamnejší bajt je na vyššej adrese
 - **big-endian** – najvýznamnejší bajt je na nižšej adrese
- Dôležité zohľadniť pri prenose dát medzi počítačmi napr. po sieti alebo cez súbory (serializácia)
- Pre programátora je pri *bežnom programovaní* transparentná.
- Väčšina bežne používaných počítačov (x86, aarch64*) využíva **little-endian**.
- **Big-endian** je využívaný v sieťovej komunikácii

Endianita – príklad

- Uvažujme typ `int` o veľkosti 4B:

$$x = 1478958_{10} = 101101001000100101110_2 = 00000000000101101001000100101110_2$$

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]
00000000	00000000	00000000	00000000	00000000	00010110	10010001	00101110	00000000	00000000	00000000	00000000	00000000

Big-endian

x

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]
00000000	00000000	00000000	00000000	00101110	10010001	00010110	00000000	00000000	00000000	00000000	00000000	00000000

Little-endian

x

Výrazy

Výrazy – literály a operátory



Výrazy – literály

- Literál je jednoduchý výraz, ide o hodnotu zapísanú priamo v kóde
- Každý výraz má typ
- `auto` – typ premennej bude automaticky určený typom inicializačného výrazu

```
auto x = 20;  
auto y = 2.71;
```
- Akého typu sú premenné `x` a `y`?

Výrazy – celočíselný literál

- Celočíselný literál je typu `int`, koncovkou je možné pridať modifikátor:
 - `20` – signed int
 - `20L` – signed long int
 - `20LL` – signed long long int
 - `20U` – unsigned int
 - `20UL` – unsigned long int
 - `20ULL` – unsigned long long int
- Od C++23:
 - `20ZU` – `std::size_t`
 - `20Z` – signed verzia `std::size_t`

Výrazy – celočíselný literál

- Číslo je tiež možné zapísať v rôznych sústavách:
 - `auto x = 179;` – desiatkovej
 - `auto y = 0xB3;` – šestnástkovej
 - `auto z = 0b10110011;` – dvojkovej
 - `auto q = 0263;` – osmičkovej
- Tiež je možné použiť znak ' ako oddelovač tisícov:
 - `auto g = 8'646'513;`

Výrazy – literál s pohyblivou desatinnou čiarkou

- Literál s pohyblivou desatinnou čiarkou je typu `double`:

```
auto x = 2.71;
```

- Koncovkou je možné zmeniť typ na:

```
auto y = 2.71F; – float
```

```
auto z = 2.71L; – long double
```

- Tiež je možné použiť vedecký zápis:

```
auto q = 0.271e1;
```

Výrazy

- Zložitejšie výrazy sa skladajú z literálov, premenných, **operátorov** a ich **operandov***
- Poznáme rôzne typy operátorov:
 - Aritmetické
 - Logické
 - Relačné
 - Inkrementačné
 - Priradovacie
 - Iné (*new*, *delete*, pretypovania, ...)

Môžu tu byť aj ďalšie veci, s ktorými sa stretneme neskôr. *

Aritmetické operátory

Všeobecne známe aritmetické operátory:

$+a$	unárne plus
$-a$	unárne mínus
$a + b$	sčítanie
$a - b$	odčítanie
$a * b$	násobenie
a / b	delenie celočíselné alebo reálne – podľa typov operandov*
$a \% b$	zvyšok po delení

Občas môže prekvapiť a spôsobiť logickú chybu. *

Aritmetické (bitové) operátory

Aritmetické bitové operátory:

<code>~a</code>	bitový not	<code>~0b00000011 == 0b11111100</code>
<code>a & b</code>	bitový and	<code>(0b11110111 & 0b00011000) == 0b00010000</code>
<code>a b</code>	bitový or	<code>(0b11000000 0b00000011) == 0b11000011</code>
<code>a ^ b</code>	bitový xor	<code>(0b11000000 ^ 0b10000011) == 0b01000011</code>
<code>a << b</code>	bitový posun vľavo	<code>(0b00100111 << 2) == 0b10011100</code>
<code>a >> b</code>	bitový posun vpravo	<code>(0b00100111 >> 2) == 0b00001001</code>

- Pri použití týchto operátorov je vo *všeobecnosti* lepšie pracovať s **unsigned** číslami

V príkladoch kvôli prehľadnosti uvažujeme 8-bitové čísla. *

Logické operátory

- Všeobecne známe logické operátory:

<code>!a</code>	logická negácia	<code>!false == true</code>
<code>a && b</code>	logický súčin (and)	<code>false && <hocičo> == false</code>
<code>a b</code>	logický súčet (or)	<code>true <hocičo> == true</code>

- Podliehajú **skráteneému vyhodnocovaniu** (angl. short-circuit evaluation)

Relačné operátory

- Všeobecne známe relačné operátory:

<code>a == b</code>	rovný
<code>a != b</code>	nerovný
<code>a < b</code>	menší
<code>a > b</code>	väčší
<code>a <= b</code>	menší alebo rovný
<code>a >= b</code>	väčší alebo rovný
<code>a <=> b</code>	trojcestné porovnanie*

Od C++20. Bude pokryté na neskoršej prednáške. *

Inkrementačné operátory

- Inkrementačné operátory:

++a	pred-inkrementácia
--a	pred-dekrementácia
a++	post-inkrementácia
a--	post-dekrementácia

- Pred vs. post:

- Pred – inkrementuje premennú a vráti **novú** hodnotu
- Post – inkrementuje premennú a vráti **pôvodnú** hodnotu

```
int main() {  
    int a = 10;  
    int b = ++a;  
    // a == 11  
    // b == 11  
}
```

```
int main() {  
    int a = 10;  
    int b = a++;  
    // a == 11  
    // b == 10  
}
```

Typové konverzie

Typové konverzie v jazyku C++



Typové konverzie

Z pohľadu zapojenia programátora rozlišujeme:

- **Implicitné** konverzie
- **Explicitné** konverzie

Z pohľadu pamäťovej reprezentácie rozlišujeme:

- Skutočnú **konverziu** – zmenu pamäťovej reprezentácie
- Zmenu **pohľadu** na pamäť (angl. reinterpretation)

Implicitné konverzie

- Početné najmä v slabo typovaných jazykoch
- Môžu uľahčovať písanie kódu
- Zároveň však môžu znižovať jeho čitateľnosť
- Môžu spôsobovať behové a logické chyby

Implicitná konverzia

```
#include <iostream>

int main() {
    int x = 314;
    double y = x; // ←
    std::cout << y << "\n";
    // 314
}
```

```
#include <iostream>

int main() {
    double y = 3.14;
    int x = y; ; // ← !!!
    std::cout << x << "\n";
    // 3 !!!
}
```

```
main:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp
    movl     $314, -4(%rbp)
    pxor     %xmm0, %xmm0
    cvtsi2sdl -4(%rbp), %xmm0 # ←
    movsd    %xmm0, -16(%rbp)
```

```
main:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp
    movsd    .LC0(%rip), %xmm0
    movsd    %xmm0, -8(%rbp)
    movsd    -8(%rbp), %xmm0
    cvtsd2sil %xmm0, %eax # ←
```

Implicitná konverzia z celého čísla na číslo s pohyblivou desatinnou čiarkou. Skutočná zmena reprezentácie.

Explicitná konverzia

- V kóde ju vyjadríme použitím niektorého z operátorov pretypovania:
 - static_cast
 - Definované konverzie medzi typmi (napr. `int` na `double`)
 - Nerobí kontrolu správnosti za behu
 - Môže meniť pamäťovú reprezentáciu
 - dynamic_cast
 - Konverzie v rámci hierarchie dedičnosti
 - Kontroluje skutočný typ za behu
 - Iba mení pohľad na pamäť

Explicitná konverzia

- V kóde ju vyjadríme použitím niektorého z operátorov pretypovania:
 - reinterpret_cast
 - Používame len výnimočne, keď **presne vieme**, čo robíme
 - Iba mení pohľad na pamäť
 - const_cast
 - Používame len výnimočne, keď **presne vieme**, čo robíme
 - Iba mení pohľad na pamäť
 - Pretypovanie v štýle jazyka C
 - Nepoužívame, vždy použijeme vhodný z vyššie uvedených

Explicitná konverzia

```
#include <iostream>

int main() {
    int timeInMs = 854;
    double timeInS = timeInMs / 1000;
    std::cout << timeInS << "\n";
    // 0 !!!
}
```

Iná možnosť ako dosiahnuť požadovaný
výsledok bez explicitného pretypovania?

```
#include <iostream>

int main() {
    int timeInMs = 854;
    double timeInS = static_cast<double>(timeInMs) / 1000;
    std::cout << timeInS << "\n";
    // 0.854
}
```

Neskôr si ukážeme lepší spôsob reprezentácie času. *

Typy v jazyku C++

Tvorba zložených typov v jazyku C++



Zložené typy (Compound types)

- Zložené typy sa skladajú z fundamentálnych
- Patria sem:
 - Pole
 - Ukazovateľ (angl. pointer)
 - Odkaz (angl. reference)
 - Enum
 - Trieda
 - Union
 - Funkcia

Pole

- Súčasť jazyka
- V *modernom** C++ je preferované použitie triedy `std::array`

```
#include <iostream>

int main() {
    int xs[3];
    xs[0] = 10;
    xs[1] = 20;
    xs[2] = xs[0] + xs[1];
    std::cout << xs[2] << "\n";
}
```

```
#include <iostream>

int main() {
    int xs[] = {10, 20, 30};
    std::cout << xs[2] << "\n";
}
```

Nech už to znamená čokoľvek... *

Enum

- Silne typované pomenované celočíselné konštanty
- Jednoduchšie ako v jazyku Java

```
#include <iostream>

enum class House { Gryffindor, Hufflepuff, Ravenclaw, Slytherin };

int main() {
    Wizard *wizard = new Wizard("Tom", House::Slytherin);
    if (wizard->getHouse() == House::Slytherin) {
        std::cout << "Shhsha Hsuhss!\n";
    }
}
```

Trieda a union

- Triedy dobre poznáte z predchádzajúcich predmetov
- Budeme sa im venovať v nasledujúcich prednáškach
- `union` je štruktúra zdedená z jazyka C, v *modernom* C++ ju nahrádza trieda `std::variant`

```
class Wizard {  
public:  
    Wizard(std::string name, House house);  
  
    std::string_view getName() const;  
    House getHouse();  
  
private:  
    std::string m_name;  
    House m_house;  
};
```

Ukazovateľ a odkaz

- Uchovávajú adresu pamäťového miesta, na ktorom sa nachádza iná premenná
- Umožňujú **nepriamo** prístupovať k inej premennej a modifikovať ju
- Spôsob nepriameho prístupu (angl. indirection)
- **Ich pochopenie je esenciálne** – bude im venovaná nasledujúca prednáška

```
#include <iostream>

int main() {
    int x = 10;
    int *px = &x;
    *px = 200;
    std::cout << x << "\n";
    // 200
}
```

```
#include <iostream>

int main() {
    int x = 10;
    int &rx = x;
    rx = 200;
    std::cout << x << "\n";
    // 200
}
```

Zhrnutie

- Základný prehľad fundamentálnych typov
 - Mať predstavu o tom, ako sú v pamäti reprezentované
- Základný prehľad operátorov
 - Poznať úskalia (celočíselné delenie, skrátené vykonávanie, ...)
- Identifikácia implicitných konverzií a ich dôsledkov
- Ak je niečo možné vyjadriť explicitne, urobte to tak

Ďakujem za pozornosť

Priestor na vaše otázky

